SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – APPLICATION BASED QUESTIONS

Course Code:	CSA17	**Course Title:**	Artificial Intelligence
CO Assessed:	CO2	**Assessment:**	Application Based Questions, In-class
Weightage:	40% of CIA	**Duration / Marks:**	60 mins
CO2 Statement:	Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)		
Student Name:	Roshan R	**Reg. No.:**	192524085
Section / Batch:	AI&DS	**Date:**	20/10/2026

Code of Conduct

I Roshan R 192524085 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *Roshan R*

Instructions

- Read each question carefully before answering (in-Class Assessment)
- Each question is a real-world application scenario. Identify the AI technique that fits, apply it step by step, and justify every decision.
- Questions are drawn from Informed Search (Greedy, A*, heuristics), Local Search and Online Search, Constraint Satisfaction Problems, and Adversarial Search (Minimax, Alpha-Beta, evaluation functions).
- Show intermediate tables (open lists, domains, α/β values) and state the final answer clearly.
- Marks will be awarded using the Application Based Questions rubric: understanding of problem context, application of concepts, problem-solving approach, accuracy of solution and clarity.
- Upload your work in GCR and Github.

Section / Topic	Focus	Question Nos.
1. Informed Search Strategies	Greedy best-first search, A* search, heuristic functions (admissibility, consistency)	**Q1**
2. Local Search, Online Search & Constraint Satisfaction	Local search and optimization problems, online search agent, CSP (MRV, degree heuristic, forward checking)	**Q2, Q3**
3. Game Playing: Adversarial Search	Games, minimax algorithm, alpha-beta pruning, evaluation functions	**Q4**

Rubrics for Application Based Questions

Criteria	Weightage	Level 4 – Exemplary (100% of marks)	Level 3 – Proficient (75%)	Level 2 – Developing (50%)	Level 1 – Beginning (25%)
Understanding of Problem Context	**20%**	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	**30%**	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	**20%**	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	**20%**	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	**10%**	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context (2)	Application of Concepts (3)	Problem-Solving Approach (2)	Accuracy of Solution (2)	Clarity (1)	Sub. Total (10)	Total %
1							
2							
3							
4							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Annexure A – Application Based Questions

Question 1: Campus Delivery Robot – Greedy Best-First vs A*

Topic: Informed search strategies, heuristic functions | 15 minutes

A campus delivery robot must carry a parcel from the Store (S) to the Hostel (G). The one-way road network and the robot's heuristic estimate h(n) of the remaining travel cost to G (in minutes) are given below.

From	To	Cost
S	A	1
S	B	4
A	C	2
A	D	5
B	G	5
C	D	1
D	G	2

Node	S	A	B	C	D	G
h(n)	6	5	3	3	2	0

Assume ties are broken alphabetically and that a node is re-opened if a cheaper path to it is found.

(a) Apply Greedy Best-First Search from S to G. Write the order in which nodes are expanded, the path returned and its total cost.

(b) Apply A* search. Show the open list with g(n), h(n) and f(n) after each expansion, the order of expansion, and the final path and cost.

(c) Check whether h(n) is admissible and consistent by verifying the required condition for every node and every road.

(d) Compare the outputs of (a) and (b). Explain why Greedy Best-First Search returned a different result and which algorithm you would deploy for the robot, with justification.

Question 2: Wi-Fi Access Point Placement – Local Search and Online Search

Topic: Local search algorithms and optimization problems, online search agent | 15 minutes

A technician wants to place a Wi-Fi access point at the best position along a corridor with 10 candidate positions x = 0 to 9. The measured signal quality f(x) (higher is better) is:

x	0	1	2	3	4	5	6	7	8	9
f(x)	2	5	9	6	4	7	12	10	8	3

A move shifts the access point to an adjacent position (x − 1 or x + 1). Steepest-ascent hill climbing moves to the better neighbour and stops when no neighbour is strictly better.

(a) Run steepest-ascent hill climbing from the start positions x = 0, x = 4 and x = 9. Show each step and the final position and signal quality.

(b) Identify all local maxima and the global maximum. If the start position is chosen uniformly at random from the 10 positions, find the probability that one run reaches the global maximum, the probability that at least one of 3 random-restart runs does, and the expected number of runs needed.

(c) A simulated annealing search is at x = 2 and proposes the move to x = 3. Compute the acceptance probability e^(Δ/T) at T = 2 and at T = 1, and explain what this does to the chance of escaping the local maximum as T falls.

(d) The same search is later done by a robot that cannot see the whole corridor, can only measure f(x) where it stands, and cannot undo exploration for free. Explain why this is an online search problem, and state what the agent must remember to avoid cycling.

Question 3: Exam Timetable Scheduling – Constraint Satisfaction Problem

Topic: Constraint satisfaction problem (MRV, degree heuristic, forward checking) | 15 minutes

Five exams A, B, C, D and E must each be given one slot from {1, 2, 3}. Two exams that share students must not be in the same slot. The pairs sharing students are: A–B, A–C, A–D, B–C, B–D, C–E, D–E. Because of invigilator availability, exam A can only be in slots 1 or 2, and exam E only in slots 1 or 2.

Exam	Initial domain
A	{1, 2}
B	{1, 2, 3}
C	{1, 2, 3}
D	{1, 2, 3}
E	{1, 2}

Rules for the search: choose the variable using Minimum Remaining Values (MRV), break ties with the degree heuristic, then alphabetically; try values in ascending order; use forward checking after every assignment.

(a) Draw the constraint graph (list its edges) and compute the degree of every exam.

(b) Select the first exam to assign using MRV and the degree heuristic, justify the choice, assign its first value, and show the domains of all exams after forward checking.

(c) Continue the search with the same rules, showing the chosen exam, its value and the updated domains at each step, until a complete assignment is found. Verify it against every constraint.

(d) Interpret: explain how MRV, the degree heuristic and forward checking reduced the search effort here, and what forward checking would signal if a domain became empty.

Question 4: Board-Game AI – Minimax, Alpha-Beta Pruning and Evaluation Function

Topic: Game playing, adversarial search, evaluation functions | 15 minutes

A game-playing agent (MAX) searches two plies ahead: MAX chooses one of three moves (B, C, D), then MIN replies with one of three moves. Positions at depth 2 are scored by the evaluation function

$$\mathbf{Eval = 2 \times M + 1 \times Mo}$$

where M is the material advantage of MAX and Mo is the mobility advantage of MAX. The leaf features (M, Mo) are given for the leaves in left-to-right order:

MIN node	Leaf 1 (M, Mo)	Leaf 2 (M, Mo)	Leaf 3 (M, Mo)
B	(1, 1)	(5, 2)	(3, 2)
C	(1, 0)	(1, 2)	(2, 2)
D	(6, 2)	(2, 1)	(0, 2)

(a) Compute the Eval value of all nine leaves.

(b) Run plain minimax. Give the backed-up value of B, C and D, the value at the root and the move MAX should play.

(c) Run alpha-beta pruning on the same tree (left to right). Show α and β at each node, mark every pruned branch, and state the root value and the number of leaves evaluated.

(d) If the leaves under C were ordered (6, 4, 2) instead, how many leaves would alpha-beta evaluate? Explain the effect of move ordering and why pruning never changes the minimax value.

Annexure B – Answers

Student Name: **Roshan R** Reg. No.: **192524085** Course: CSA17 – Artificial Intelligence

Answer 1: Campus Delivery Robot – Greedy Best-First vs A*

Given: roads S → A (1), S → B (4), A → C (2), A → D (5), B → G (5), C → D (1), D → G (2). h(S)=6, h(A)=5, h(B)=3, h(C)=3, h(D)=2, h(G)=0.

(a) Greedy Best-First Search

Greedy search always expands the node with the smallest h(n) and ignores the cost already travelled.

Step	Node expanded	Children added (h)	Open list after expansion (sorted by h)
1	S	A (5), B (3)	B(3), A(5)
2	B (lowest h = 3)	G (0)	G(0), A(5)
3	G (h = 0)	Goal reached	–

Order of expansion: S, B, G. **Path returned:** S → B → G. **Cost** = 4 + 5 = **9 minutes**. Greedy is not optimal: it took the path with the lowest h at each step, even though B → G is expensive.

(b) A* Search, f(n) = g(n) + h(n)

Step	Expanded	Open list after expansion: node (g, h, f)
0	–	S (0, 6, 6)
1	S	A (1, 5, 6), B (4, 3, 7)
2	A (f = 6)	C (3, 3, 6), B (4, 3, 7), D (6, 2, 8)
3	C (f = 6)	D (4, 2, 6), B (4, 3, 7) [D reached via C: g = 3 + 1 = 4 < 6, so D's entry is updated; old D (6, 2, 8) removed]
4	D (f = 6)	G (6, 0, 6), B (4, 3, 7)
5	G (f = 6)	Goal reached. Remaining open list: B (4, 3, 7)

Order of expansion: S, A, C, D, G. **Final path:** S → A → C → D → G. **Cost** = 1 + 2 + 1 + 2 = **6 minutes**. B (f = 7) was never expanded because its f is larger than the goal's f = 6.

(c) Admissibility and consistency of h(n)

Admissible means h(n) ≤ h*(n), where h*(n) is the true cheapest cost from n to G. Computing h* backwards from G:

Node	True cost h*(n)	Cheapest path	h(n)	h(n) ≤ h*(n)?
G	0	–	0	Yes
D	2	D → G = 2	2	Yes (2 ≤ 2)
C	3	C → D → G = 1 + 2	3	Yes (3 ≤ 3)
B	5	B → G = 5	3	Yes (3 ≤ 5)
A	5	A → C → D → G = 2+1+2 (A → D → G = 7)	5	Yes (5 ≤ 5)
S	6	S → A → C → D → G = 1+2+1+2	6	Yes (6 ≤ 6)

Consistent means h(n) ≤ c(n, n′) + h(n′) for every road n → n′:

Road	h(n)	c(n, n′) + h(n′)	Holds?
S → A	6	1 + 5 = 6	Yes (6 ≤ 6)
S → B	6	4 + 3 = 7	Yes (6 ≤ 7)

A → C	5	2 + 3 = 5	Yes (5 ≤ 5)
A → D	5	5 + 2 = 7	Yes (5 ≤ 7)
B → G	3	5 + 0 = 5	Yes (3 ≤ 5)
C → D	3	1 + 2 = 3	Yes (3 ≤ 3)
D → G	2	2 + 0 = 2	Yes (2 ≤ 2)

Also h(G) = 0. Every node and every road satisfies the condition, so **h(n) is both admissible and consistent**. The f-values along the expanded path never decrease (6, 6, 6, 6, 6), which confirms consistency.

(d) Comparison and recommendation

	Greedy Best-First	A*
Nodes expanded	S, B, G (3)	S, A, C, D, G (5)
Path	S → B → G	S → A → C → D → G
Total cost	9 min	6 min (optimal)

Greedy search returned a different result because it ranks nodes only by h(n). At S, B looked closest to the hostel (h = 3 against 5 for A), so it went to B, and the expensive road B→G (cost 5) was only discovered after it had committed. It never compares the real cost already spent (g) and so it cannot recover. A* uses f = g + h, so A's path (f = 6) stays cheaper than B (f = 7) until G is reached with a total of 6.

Recommendation: I would deploy A*. Since h is admissible and consistent, A* is guaranteed to return the least-cost route (6 min against 9 min, a 33% saving) and never needs to re-open a node. Its cost is only two extra expansions on this small map. Greedy is faster but gives no optimality guarantee, so it is only suitable when a rough route is enough.

Answer 2: Wi-Fi Access Point Placement – Local Search and Online Search

Given:

x	0	1	2	3	4	5	6	7	8	9
f(x)	2	5	9	6	4	7	12	10	8	3

(a) Steepest-ascent hill climbing

At each step the agent looks at both neighbours (x−1, x+1), moves to the better one only if it is strictly better than the current value, otherwise it stops.

Start	Step	Current x (f)	Neighbours (f)	Decision
x = 0	1	0 (2)	1 (5)	Move to 1 (5 > 2)
	2	1 (5)	0 (2), 2 (9)	Move to 2 (9 is best)
	3	2 (9)	1 (5), 3 (6)	No better neighbour. STOP
	Result	**x = 2, f = 9**		Local maximum, not global
x = 4	1	4 (4)	3 (6), 5 (7)	Move to 5 (7 > 6)
	2	5 (7)	4 (4), 6 (12)	Move to 6 (12)
	3	6 (12)	5 (7), 7 (10)	No better neighbour. STOP
	Result	**x = 6, f = 12**		Global maximum
x = 9	1	9 (3)	8 (8)	Move to 8 (8 > 3)
	2	8 (8)	7 (10), 9 (3)	Move to 7 (10)

	3	7 (10)	6 (12), 8 (8)	Move to 6 (12)
	4	6 (12)	5 (7), 7 (10)	No better neighbour. STOP
	Result	**x = 6, f = 12**		Global maximum

(b) Local maxima, global maximum and restart probabilities

A position is a local maximum if it is higher than both neighbours (an end position needs to be higher than its single neighbour). x = 2 (9 > 5 and 6) and x = 6 (12 > 7 and 10) qualify. The ends fail: f(0)=2 < f(1)=5 and f(9)=3 < f(8)=8. So the **local maxima are x = 2 (f = 9) and x = 6 (f = 12)**, and the **global maximum is x = 6, f = 12**.

Starting positions and where they end up: x = 0, 1, 2, 3 climb to x = 2 (for x = 3, the neighbour 2 has f = 9 against 4 for x = 4). x = 4, 5, 6, 7, 8, 9 climb to x = 6. So 6 of the 10 starts reach the global maximum.

- P(one run reaches global maximum) = 6/10 = **0.6**
- P(at least one of 3 random-restart runs succeeds) = $1 - (0.4)^3 = 1 - 0.064 =$ **0.936**
- Expected number of runs until the first success (geometric distribution) = 1/p = 1/0.6 = **5/3 ≈ 1.67 runs**

(c) Simulated annealing at x = 2

Current f(2) = 9, proposed f(3) = 6, so Δ = 6 − 9 = −3 (a downhill move). Acceptance probability = $e^{\Delta/T}$.

Temperature T	Δ / T	Acceptance probability $e^{\Delta/T}$
T = 2	−3 / 2 = −1.5	$e^{-1.5} \approx 0.223$ (about 22%)
T = 1	−3 / 1 = −3	$e^{-3} \approx 0.050$ (about 5%)

When T falls from 2 to 1, the chance of accepting this bad move drops from about 22% to 5%. At high T the search accepts downhill moves often, so it can walk down from the local peak at x = 2 through x = 3 and x = 4 and then climb to x = 6. As T keeps falling, the probability of accepting a downhill move tends to 0 and annealing behaves like plain hill climbing, getting stuck at whatever peak it is on. So the chance of escaping the local maximum decreases as the temperature falls, and a slow cooling schedule is needed to give enough time to escape before it freezes.

(d) Online search

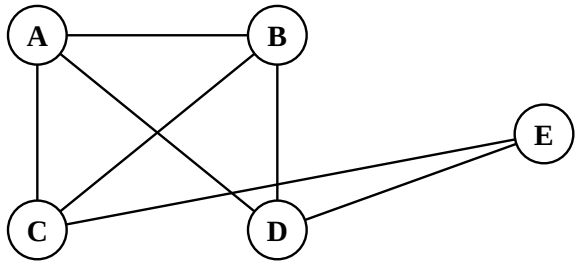
This is an **online search problem** because the robot does not know the map in advance. It can only read f(x) at the place where it stands, it learns what a move leads to only by actually making it, and every move costs real effort (it cannot be undone for free). Planning and acting are therefore interleaved, unlike offline search where the whole space is known first and a complete plan is computed before moving.

To avoid cycling (moving back and forth between the same positions) the agent must remember: (1) a **table of visited positions** with the f value measured at each one (the result of each action taken from it); (2) the **untried actions** (left/right) at each visited position, so it does not repeat a move; (3) the **list of positions it came from** so it can backtrack when every action at a position has been tried; and (4) its **current position** and the best value found so far. Because the corridor is reversible (a move left can be undone by a move right), a depth-first style online search with backtracking, or LRTA* which updates a stored estimate for each visited cell, will explore it without looping forever.

Answer 3: Exam Timetable Scheduling – Constraint Satisfaction Problem

Variables: A, B, C, D, E. **Domains:** A {1,2}, B {1,2,3}, C {1,2,3}, D {1,2,3}, E {1,2}. **Constraints:** exams in each pair share students and so must differ in slot: A≠B, A≠C, A≠D, B≠C, B≠D, C≠E, D≠E.

(a) Constraint graph and degrees



Exam	Neighbours	Degree
A	B, C, D	3
B	A, C, D	3
C	A, B, E	3
D	A, B, E	3
E	C, D	2

Edges (7): A–B, A–C, A–D, B–C, B–D, C–E, D–E. Sum of degrees = 3+3+3+3+2 = 14 = 2 × 7 edges, which checks out.

(b) First variable using MRV and degree heuristic

Number of remaining values: A = 2, B = 3, C = 3, D = 3, E = 2. MRV picks the smallest domain, so A and E are tied at 2 values. The tie is broken by the degree heuristic (the variable involved in most constraints): deg(A) = 3 > deg(E) = 2, so **A is chosen first**. A is the better choice because fixing it prunes the domains of three neighbours (B, C, D) at once.

Assign **A = 1** (ascending order). Forward checking removes 1 from the neighbours B, C and D:

Exam	Before	After A = 1 (forward checking)
A	{1, 2}	{1} (assigned)
B	{1, 2, 3}	{2, 3}
C	{1, 2, 3}	{2, 3}
D	{1, 2, 3}	{2, 3}
E	{1, 2}	{1, 2} (not a neighbour of A, unchanged)

(c) Continuing the search

For the later steps, the degree is counted among the exams that are still unassigned.

Step	Domains before	Choice (reason)	Value	Domains after forward checking
2	B{2,3} C{2,3} D{2,3} E{1,2}	B. MRV: all four have 2 values. Degree among unassigned: B 2, C 2, D 2, E 2, still tied. Alphabetical gives B	B = 2	A{1} B{2} C{3} D{3} E{1,2} (2 removed from C and D)
3	C{3} D{3} E{1,2}	C. MRV: C and D have 1 value. Degrees tied (1 each), alphabetical gives C	C = 3	A{1} B{2} C{3} D{3} E{1,2} (3 is not in the domain of E, so E is unchanged)
4	D{3} E{1,2}	D. MRV: D has 1 value, E has 2	D = 3	E{1,2} (unchanged)
5	E{1,2}	E, the only one left	E = 1 (first value, valid)	E{1}. All exams assigned

Complete assignment: A = 1, B = 2, C = 3, D = 3, E = 1. No backtracking was needed.

Verification against all constraints:

Constraint	Slots	Satisfied?
A ≠ B	1 ≠ 2	Yes
A ≠ C	1 ≠ 3	Yes
A ≠ D	1 ≠ 3	Yes
B ≠ C	2 ≠ 3	Yes
B ≠ D	2 ≠ 3	Yes
C ≠ E	3 ≠ 1	Yes
D ≠ E	3 ≠ 1	Yes
A in {1,2}	A = 1	Yes
E in {1,2}	E = 1	Yes

C and D are in the same slot (3), which is allowed because C and D do not share students (C–D is not an edge).

(d) Interpretation

MRV selected the most constrained exam first (A, with only slots 1 and 2 available), which is the one most likely to cause a failure. Handling it early means a wrong choice shows up quickly and not deep in the tree. **The degree heuristic** broke the tie between A and E by choosing A, which touches three other exams, so its assignment removes the most options from the rest. **Forward checking** removed the used slot from the neighbours after every assignment. After A = 1 and B = 2, the domains of C and D shrank to the single value {3}, so the later steps were forced and no guessing was needed. The result was only five assignments with zero backtracks, against up to 2 × 3 × 3 × 3 × 2 = 108 complete assignments for blind enumeration.

If a domain became empty, forward checking would signal a dead end straight away: no legal slot remains for that exam, so the current partial assignment cannot be completed. The search would then immediately undo the latest assignment (backtrack) and try the next value, or go back further if that value is also exhausted, without wasting time on the remaining variables.

Answer 4: Board-Game AI – Minimax, Alpha-Beta Pruning and Evaluation Function

(a) Evaluation of the leaves, Eval = 2 × M + 1 × Mo

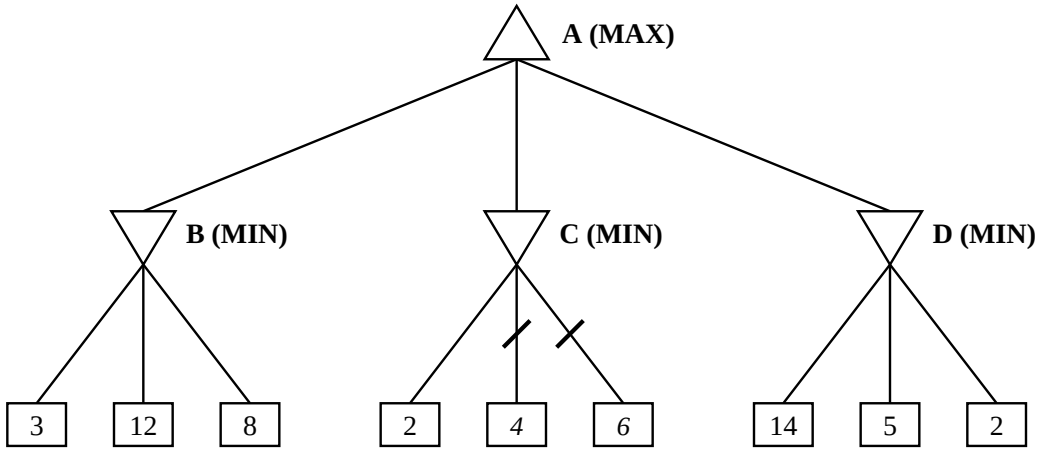
MIN node	Leaf	(M, Mo)	Calculation	Eval
B	1	(1, 1)	2×1 + 1 = 3	3
B	2	(5, 2)	2×5 + 2 = 12	12
B	3	(3, 2)	2×3 + 2 = 8	8
C	1	(1, 0)	2×1 + 0 = 2	2
C	2	(1, 2)	2×1 + 2 = 4	4
C	3	(2, 2)	2×2 + 2 = 6	6
D	1	(6, 2)	2×6 + 2 = 14	14
D	2	(2, 1)	2×2 + 1 = 5	5
D	3	(0, 2)	2×0 + 2 = 2	2

(b) Plain minimax

MIN nodes take the minimum of their leaves and the root (MAX) takes the maximum of its children.
B = min(3, 12, 8) = **3** C = min(2, 4, 6) = **2** D = min(14, 5, 2) = **2**

Root = max(3, 2, 2) = **3**. MAX should play **move B**. All 9 leaves are evaluated.

(c) Alpha-beta pruning (left to right)



Leaf values are Eval = 2M + Mo. A cross (x) on a branch marks a pruned branch; leaves in italics are never evaluated.

Node	Visited leaf	α	β	Action / value
A (root)	–	−∞	+∞	Start
B	3	−∞	+∞ → 3	β = 3, continue
B	12	−∞	3	12 is not less than 3, stays 3
B	8	−∞	3	stays 3. B returns **3**
A (root)	–	3	+∞	α updated to 3
C	2	3	+∞ → 2	β = 2 ≤ α = 3: **cut-off**. Leaves (1,2) = 4 and (2,2) = 6 **pruned**. C returns ≤ 2
D	14	3	+∞ → 14	β = 14, continue
D	5	3	14 → 5	β = 5 > α = 3, continue
D	2	3	5 → 2	β = 2 ≤ α = 3. D returns 2 (no leaves left to prune)
A (root)	–	3	+∞	max(3, 2, 2) = **3**

Root value = 3, best move is B (same as plain minimax). **Leaves evaluated = 7** (3 under B + 1 under C + 3 under D). **Pruned: 2 leaves**, both under C (Eval 4 and 6).

(d) Effect of move ordering

If the leaves under C were ordered (6, 4, 2), then at C: first leaf 6 gives β = 6 > α = 3, no cut; second leaf 4 gives β = 4 > 3, no cut; third leaf 2 gives β = 2 ≤ 3, but it is already the last leaf. B and D are unchanged (3 leaves each). So alpha-beta evaluates **3 + 3 + 3 = 9 leaves**, the same as plain minimax, and nothing is pruned.

Effect of move ordering: alpha-beta prunes best when the best (for MIN, the lowest) replies are examined first. In the original order (2, 4, 6) the lowest leaf came first, which gave an immediate cut-off and saved 2 leaves. In the order (6, 4, 2) the lowest leaf comes last, so there is no saving. With perfect ordering, alpha-beta needs about $b^{d/2}$ leaves instead of b^{d} (here the best case would be 3 + 3 − 1 = 5 leaves), so good ordering (for example trying captures or the previously best move first) lets the search go about twice as deep in the same time.

Why pruning never changes the minimax value: a branch is cut only when it is certain not to matter. When $\beta \leq \alpha$ at a MIN node, MIN can force a value of at most β, and MAX already has a different option worth at least α, so MAX will never choose that node whatever the remaining leaves are. The pruned leaves therefore cannot change the value backed up to the root, only the amount of work done. That is why both algorithms returned 3 and the move B.